

Usage

NASA EXOPLANET SEARCH

Usage:
./nasasearch --select <Column...> [--sort <Column> <Ascending|Descending>]
Options:
--select
 Required. Specifies one or more columns to return.
--sort
 Optional. Specifies the column to sort by and the sort direction.
Available columns:
 PlanetName
 HostStar
 OrbitalPeriodDays
 DistanceParsecs
Sort directions:
 Ascending
 Descending
Example: ./nasasearch --select PlanetName DistanceParsecs --sort DistanceParsecs Ascending

PlanetRecord.fs

Planet Record: Defines F# record (Similar to C++ struct) called **Planet**. The record has four fields: Name (string), HostStar (string), OrbitalPeriodDays (float), DistanceParsecs (float).

The JsonPropertyName attributes map the F# fields to the corresponding property name in the JSON data returned by NASA’s API

ArgHandler.fs

This file handles the passed user arguments. A simple program has been implemented to demonstrate how the arguements are passed.

Ideally the function should take the args (array of strings) from Main and return a special value: **Result <Query, string>** Essentially, this return value returns a **Query record** if the program can successfully verify the passed arguments, or an **Error string** if the program is unable to verify the passed arguments (Hence <Query, string>)

I would probably implement helper functions for each part of the **Query record** (recall **Query.fs Lines 10-19** for **Query** records)

You could set values to the helper functions and then once all the values are obtained successfully initialize a Query record and return it

<https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/results>

Query.fs

The Query file is used to generate a query in ADQL (Astronomical Data Query Language, specialized variant of SQL)

Lines 10 - 29
To simplify UX, discriminated Unions are used to established a list of acceptable **Columns** and acceptable **sort direction** (Ascending | Descending)

Then a record is defined (**Query**) that has fields **Select** (which is a list of values, the **columns** chosen by user) and **SortBy** (which is a tuple of a single **Column** type and a **sort direction**).

Select indicates which columns will be shown. if user selects columns Hoststar and DistanceParsecs it will show the Name (which is mandatory) and the selected columns (which are optional)

SortBy indicates the sort parameter and the direction. The user may choose to sort the data by distanceParsecs indicates descending order providing the nearest/farthest planets to earth

Lines 32 - 50
After the unions and records have been established a fuction, **buildSelect** (which takes a list of columns and uses type matching to determine if list is empty) will map the users selection to the respective NASA Table Column Name, using helper functionn **columnName** (If the user chooses to see the DistanceParsecs the helper function changes it to “sy_dist” for the ADQL request and the **buildSelect** function formats the rest to match with rest of SELECT clause for ADQL)

Lines 53 - 63
The next function **buildSortBy** does something similar for the **ORDER BY** section of the ADQL request. Takes the tuple from the user argument and builds the query based on the sortby column and the sort direction

Remainder of Program
A function puts together the query clauses created and does some magic to send a request to NASA TAP and get data back. This data is stored in an array of type **Planet**

Main.fs

The main function passes the Arguments into **parseArguments** function defined in ArgHandler.fs (**not yet implemented**)

The function should take an array of strings (the argument array) and return Result<Query, string> (explained more in ArgHandler.fs)

If **parseArguments** is successfully it should return a **Query record** (defined in **Query.fs** see **Lines 10-29**) with Select defined as the users desired columns (Hostname, DistanceParsecs) and, optionally, the column to sort by and the sort direction as a tuple (sy_dist, Desceding)

Main.fs should then pass the **Query record** into the **fetchPlanets** function (see **Remainder of Program** in **Query.fs**). This will return an array of type **Planet**.

Then the program uses simple printfn statements to output the correct data